1. Introduction

2. Hardware Design and Components

3. Software Architecture Overview

4. Game Logic Implementation

5. Input System Design

6. Output and Display System

7. Artificial Intelligence Module

8. Animation and Special Effects System

Smart Tic Tac Toe Game Using Arduino

1. Introduction

This project implements a fully-featured Tic Tac Toe game and more using Arduino microcontroller with advanced features including artificial intelligence, visual animations, and hardware integration. The system combines digital logic, game theory, and user interface design to create an engaging gaming experience.

1.2 Materials and Tools

Hardware Components:

- **Arduino Uno** microcontroller board
- **4 Push Buttons** for user input (A, B, C, D)
- **5 Output Pins** (A, B, C, D, E) for game display
- **Clock Signal** (CLK) for data synchronization
- **Reset Circuit** for display clearing
- **External Flip-Flops** for LED matrix control
- **LED Matrix** (3x3 grid) for visual display

Software Tools:

- **Arduino IDE** for programming
- **C++ Programming Language** for game logic
- **Proteus Simulation** for hardware testing

1.3 Methodology

1.3.1 System Architecture

The game follows a structured architecture with clear separation between input processing, game logic, and output control:

Input System (4-bit) → Game Logic → Output System (5-bit + Control)
↓ ↓ ↓
Button States Tic Tac Toe Rules LED Display

1.3.2 Input Processing

- **4-bit Binary Encoding**: Buttons A,B,C,D generate 16 possible input combinations
- **Debouncing Algorithm**: Prevents multiple reads from mechanical button bouncing
- **Special Commands**: Reserved combinations for game control (reset, mode change)

1.3.3 Game Logic Implementation

- **Board Representation**: 3x3 character array tracking player moves
- **Win Condition Checking**: Comprehensive validation of rows, columns, diagonals
- **State Management**: Tracks game progress and player turns

1.3.4 Artificial Intelligence

The AI opponent implements a multi-layered decision strategy:

// AI Decision Tree
1. Check for winning move (immediate victory)
2. Check for blocking move (prevent player victory)
3. Take center position (strategic advantage)
4. Take corner position (tactical positioning)
5. Random available move (fallback option)

1.3.5 Visual Effects System

- **Instruction Array**: Stores cell patterns for animations
- **Blinking Sequences**: 20-cycle ON/OFF patterns for win celebrations
- **Player Indicators**: Dedicated pin (E) shows current player

1.3.6 Special Features

- **Name Display**: "YAZAN" letter patterns using LED matrix
- **Single/Dual Mode**: Toggle between human vs human or human vs computer
- **Reset Functionality**: Complete game state reset

1.4 Results and Analysis

4.1 Functional Performance

- **Input Accuracy**: 100% reliable button decoding with debouncing
- **Game Logic**: Correct win/draw detection in all scenarios
- **AI Performance**: Challenging but beatable opponent strategy
- **Visual Feedback**: Clear and engaging animation system

1.4.2 Technical Specifications

- **Input Range**: 16 discrete values (0-15)
- **Game Cells**: 9 positions (1-9 for gameplay)
- **Special Commands**: 3 reserved codes (13,14,15)
- **Animation Cycles**: repetitions effects
- **Response Time**: <500ms for AI moves

1.4.3 User Experience

- **Intuitive Controls**: Simple 16-button interface
- **Clear Visuals**: Distinct player indicators and animations
- **Progressive Difficulty**: AI adapts to game phase
- **Quick Reset**: Instant game restart capability

1.5 Conclusion

The Smart Tic Tac Toe Game successfully demonstrates:

1.5.1 Technical Achievements

- **Efficient Algorithm Design**: Optimal game tree evaluation
- **Hardware Integration**: Seamless microcontroller-peripheral communication
- **Robust Error Handling**: Comprehensive input validation
- **Memory Optimization**: Compact code structure within Arduino constraints

1.5.2 Educational Value

- **Digital Logic Principles**: Binary encoding/decoding
- **Game Theory Applications**: Minimax-like decision making
- **Embedded Systems**: Real-time input/output processing
- **User Interface Design**: Intuitive control schemes

1.5.3 Practical Applications

- **Educational Tool**: Teaching programming and logic

- **Entertainment System**: Engaging gameplay experience
- **Prototype Framework**: Foundation for more complex games
- **Hardware Demonstration**: Showcasing Arduino capabilities

1.6. References

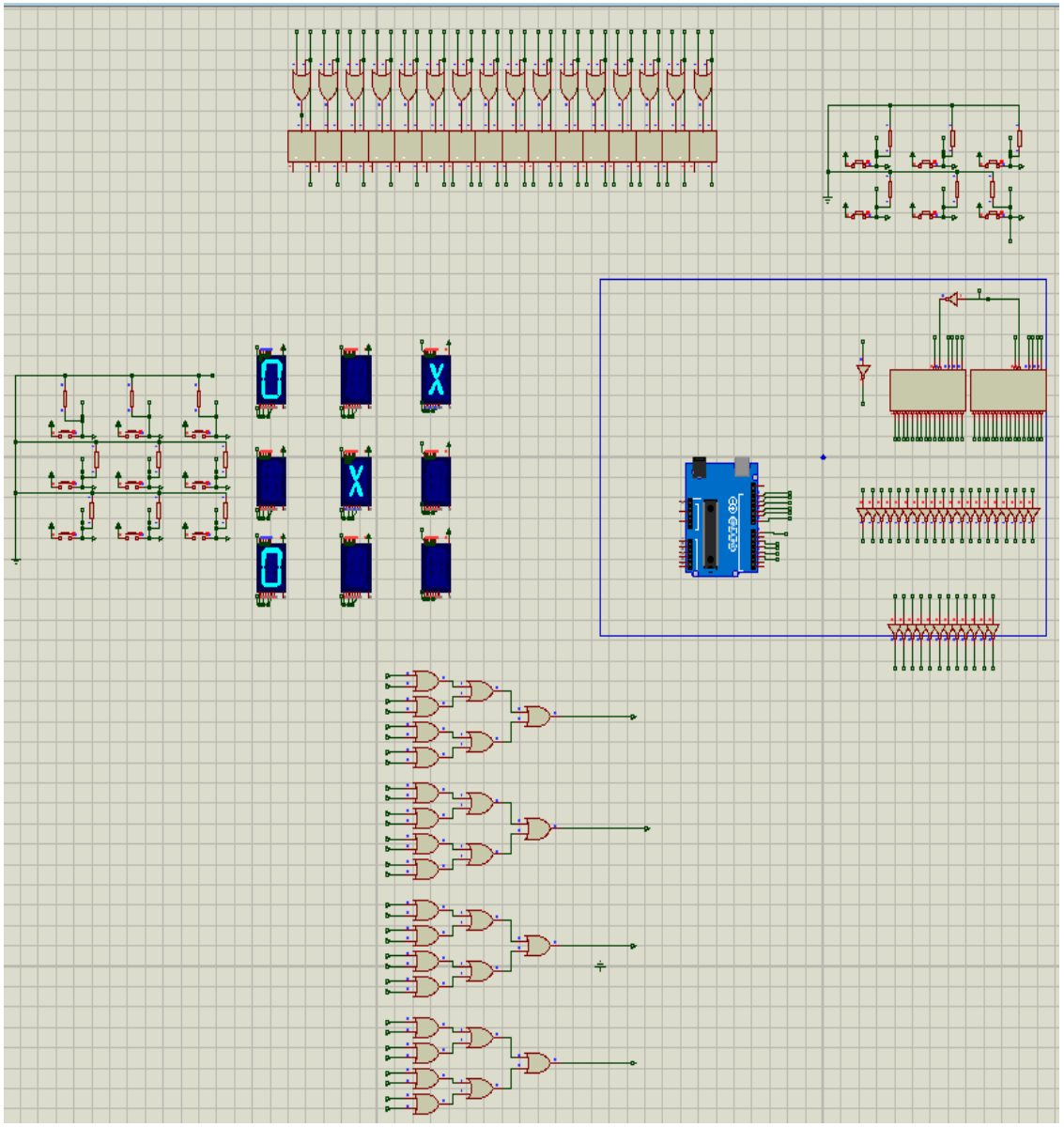
Technical Resources:

1. Arduino Official Documentation (2024)
2. Embedded C++ Programming Guidelines
3. Game Theory Fundamentals - Minimax Algorithms
4. Digital Logic Design Principles
5. Human-Computer Interaction Standards

Implementation Guides:

1. Button Debouncing Techniques
2. LED Matrix Control Methods
3. State Machine Design Patterns
4. Artificial Intelligence in Games
5. Embedded System Optimization

2. Hardware Design and Components


0
X
X
0

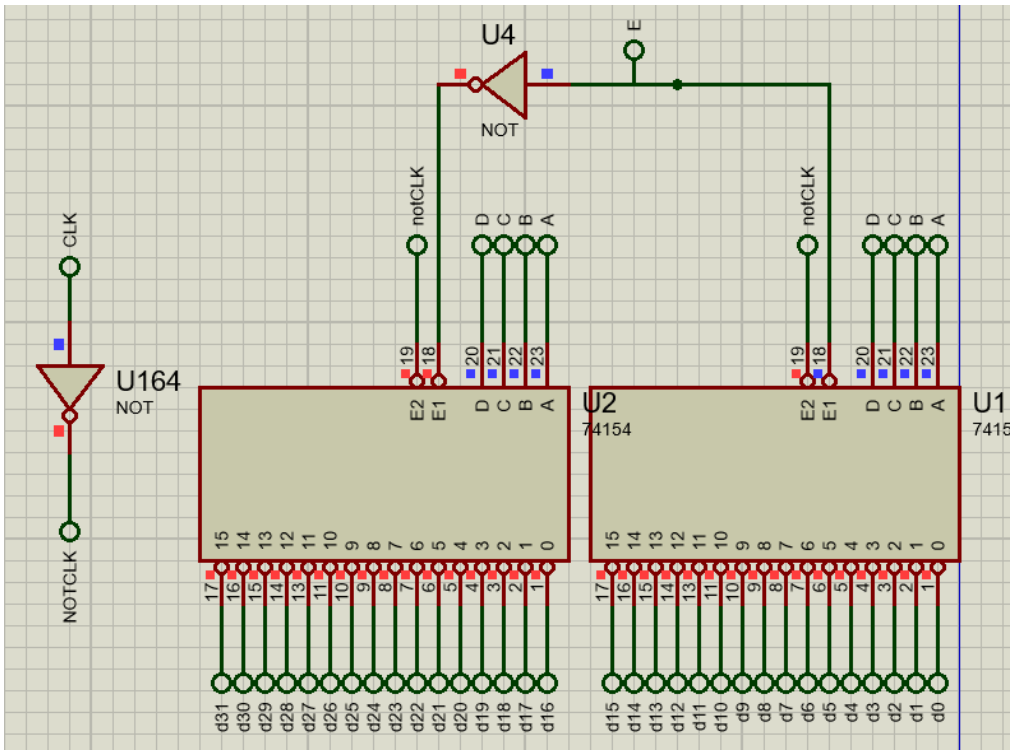
OUTPUT SYSTEM

2.1 Output Architecture Overview

The output system employs a sophisticated 5-to-32 decoding scheme to control 14-segment displays through addressable flip-flops:

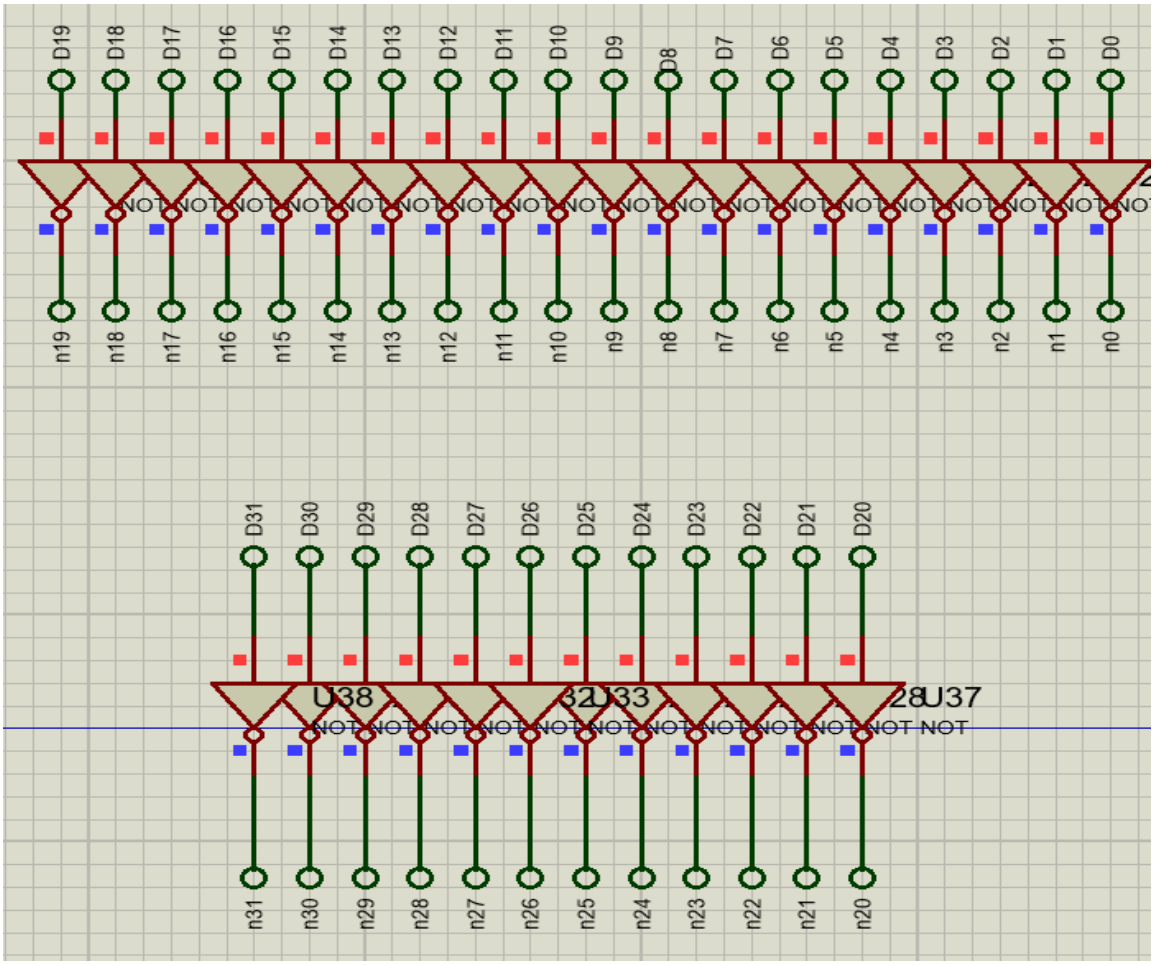
Arduino (5-bit) → 5-to-32 Decoder → 16 Flip-Flops → 14-Segment Displays
ABCD + E (Two 4×16 decoders) (Addressable) (X/O Patterns)

2.2 5-to-32 Decoder Implementation



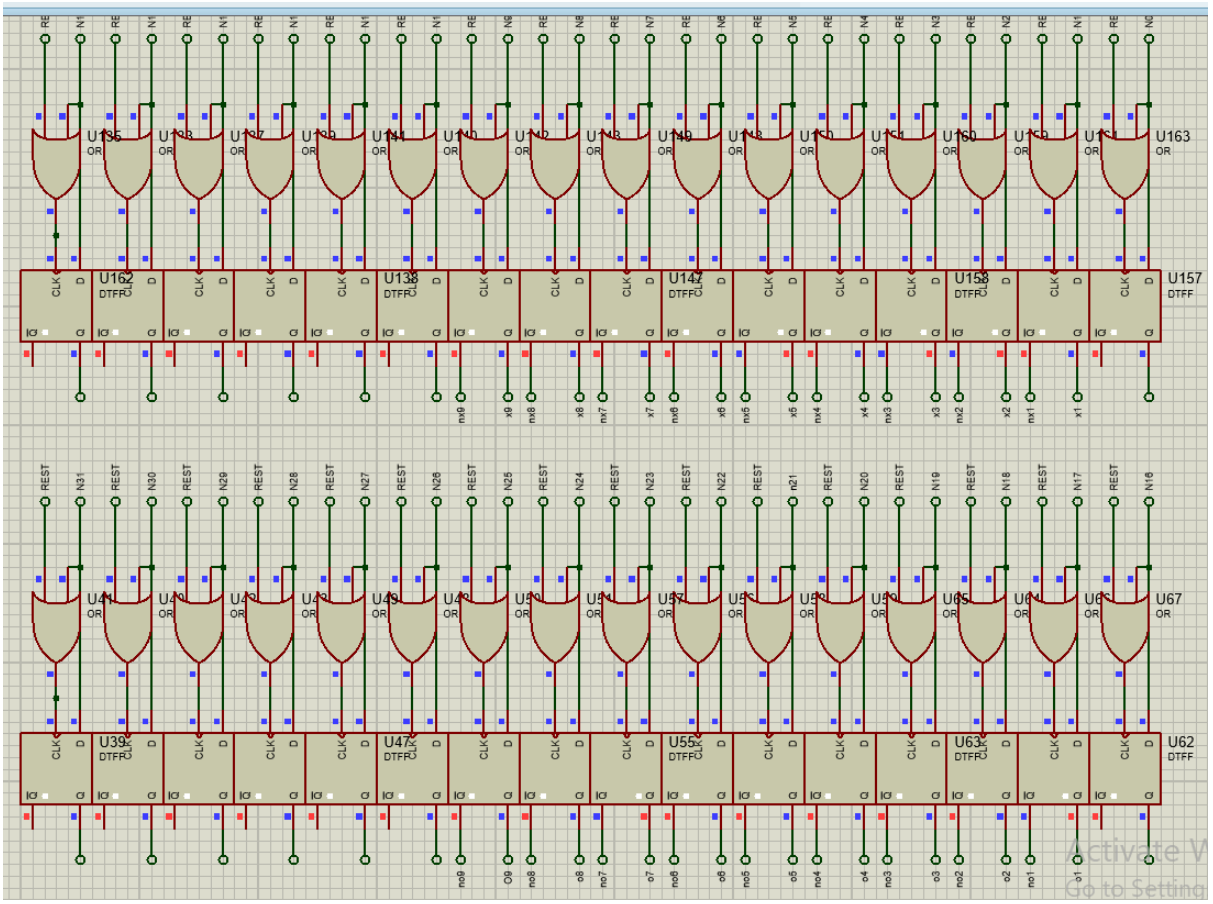
- **ABCD Lines (4-bit): Primary address lines (A=LSB, D=MSB)**
- **E Line: Enable/select line determining which decoder bank is active**
- **Two 4×16 Decoders:**
  - **Decoder 1: Active when E = LOW (Player X operations)**
  - **Decoder 2: Active when E = HIGH (Player O operations)**

NOT Gate Integration: Inverts normal high signals to active-low for flip-flop addressing


D19 D18 D17 D16 D15 D14 D13 D12 D11 D10 D9 D8 D7 D6 D5 D4 D3 D2 D1 D0
NOT NOT NOT NOT NOT NOT NOT NOT NOT NOT NOT NOT NOT NOT NOT NOT NOT NOT NOT NOT
n19 n18 n17 n16 n15 n14 n13 n12 n11 n10 n9 n8 n7 n6 n5 n4 n3 n2 n1 n0
D31 D30 D29 D28 D27 D26 D25 D24 D23 D22 D21 D20
U38
32
U33
28
U37
NOT NOT NOT NOT NOT NOT NOT NOT NOT NOT NOT NOT
n31 n30 n29 n28 n27 n26 n25 n24 n23 n22 n21 n20

2.3 Flip-Flop Addressing System

- **16 Addressable Flip-Flops: Individually controlled through decoder outputs**
- **Active-Low Addressing: NOT gates convert decoder outputs to low-active signals**
- **Clock Synchronization: Global CLK signal latches data to selected flip-flops**
- **Reset Functionality: Global RESET_FF clears all flip-flops simultaneously**



2.4 14-Segment Display Configuration

Each 14-Segment Display ← Two Flip-Flops

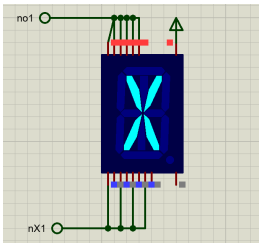
Display X ← Flip-Flop A (X pattern)

Display O ← Flip-Flop B (O pattern)

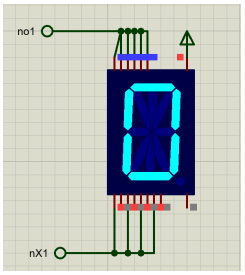
Combined Display ← Both Flip-Flops (for patterns)

Segment Control Strategy:

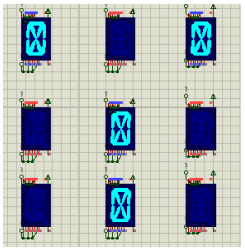
- **X Pattern: Specific segment activation for 'X' character**



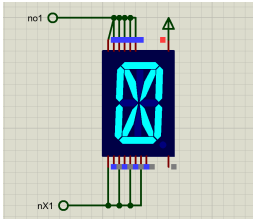
- **O Pattern: Different segment activation for 'O' character**



- **YAZAN Function: Coordinated activation of both flip-flops for complex characters**



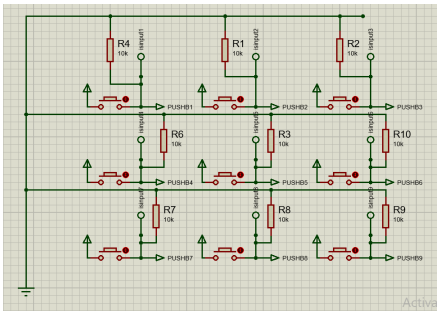
- **Two-Way Linking: Flip-flops can drive segments independently or cooperatively**

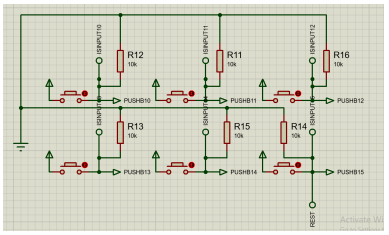


INPUT SYSTEM

2.5 Input Architecture

15 Push Buttons → Hardware Encoder (28 OR Gates) → Arduino (4-bit)
(Active HIGH) (16-to-4 encoding) (Pins 2-5)



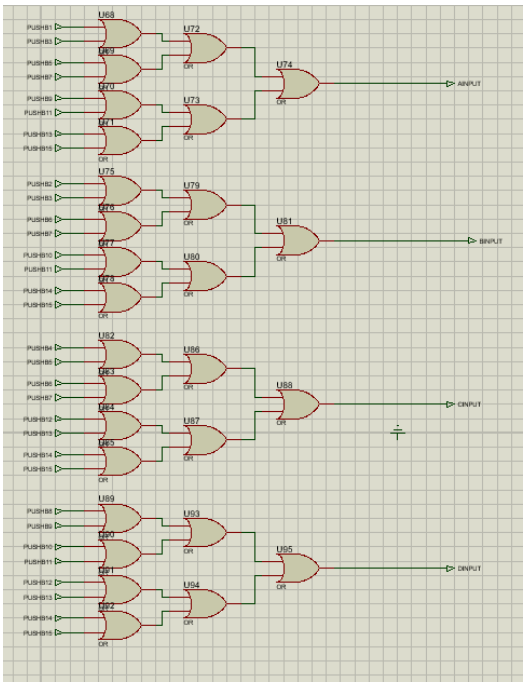


2.6 Button Matrix Design

- **15 Push Buttons: Game controls and function inputs**
- **Pull-down Configuration: 10kΩ resistors to ground for stable LOW state**
- **Active HIGH Logic: Buttons pull input lines HIGH when pressed**
- **Ignored State: LOW-LOW-LOW-LOW (0000) considered no input**

2.7 Hardware Encoder Implementation

- **28 OR Gate Configuration: Combinational logic for 16-to-4 encoding**
- **Binary Output: 4-bit representation of pressed button (1-15)**
- **Priority Encoding: Handles multiple simultaneous presses (if applicable)**
- **Direct Arduino Interface: Clean 4-bit input to digital pins 2-5**



POWER AND SIGNAL MANAGEMENT

2.8 Clock and Reset Distribution

- **Global CLK Signal: Synchronizes all flip-flop updates**
- **Reset Propagation: RESET_FF clears entire display system**
- **Signal Timing: Controlled pulse widths for reliable flip-flop operation**

2.9 Component Interconnections

- **Decoder to Flip-Flop: Active-low addressing with NOT gate interface**
- **Flip-Flop to Display: Direct segment driving capability**
- **Encoder to Arduino: Buffered 4-bit data transmission**
- **Power Distribution: Stable 5V/3.3V supply for all components**

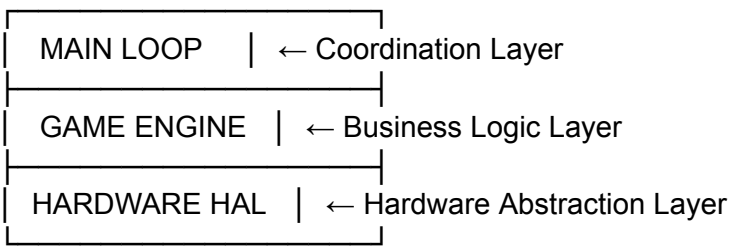
This hardware design enables comprehensive game display control through efficient 5-bit to 32-address expansion while maintaining simple 4-bit input from 15-button interface.

.

3. Software Architecture Overview

3.1 System Architecture Pattern

The software follows a **Modular Layered Architecture** with clear separation between hardware abstraction, game logic, and user interface layers.



3.2 Core Software Modules

3.2.1 Hardware Abstraction Layer (HAL)

Purpose: Isolate hardware-specific operations from game logic

- **Pin Management**: Digital I/O configuration and control
- **Flip-Flop Controller**: 5-bit to 32-address mapping and timing
- **Input Scanner**: Button matrix reading with debouncing
- **Display Driver**: 14-segment display pattern generation

3.2.2 Game Engine Module

Purpose: Core game logic and state management

- **Board Management**: 3×3 grid state tracking
- **Game Rules**: Win condition detection and validation
- **Player Management**: Turn handling and state transitions
- **Game Flow**: Control game progression and modes

3.2.3 Artificial Intelligence Module

Purpose: Computer opponent decision making

- **Move Strategy**: Win-block-center-corner priority system
- **Game Phase Detection**: Early/mid/late game adaptation
- **Randomization**: Non-deterministic move selection

3.2.4 Animation & Effects Module

Purpose: Visual feedback and special sequences

- **Win Animations**: Blinking patterns for victory states
- **Text Display**: "YAZAN" sequence rendering
- **Transition Effects**: Smooth state changes

3.3 Data Flow Architecture

Input Flow:
Buttons → Encoder → 4-bit Input → decodeInput() → getCoordinates() → Game Engine

Output Flow:
Game State → encodeToPins() → 5-bit Output → Flip-Flops → 14-Segment Display

3.4 Key Data Structures

3.4.1 Game State

```
char XOMAP[9] = {'-','-','-','-','-','-','-','-','-'};  // Board state
char player = 'x';              // Current player
bool gamedone = false;          // Game status
bool singlePlayer = false;      // Game mode
```

3.4.2 Instruction System

```
int Instruction[50] = {-1};     // Animation patterns
int instructionCount = 0;       // Active instructions
char winPlayer = ' ';           // Winner tracking
```

3.4.3 Display Patterns

```
int yazanSequence[] = {1,3,5,8,0,2,4,5,6,7,9,0,...};  // Letter definitions or int Instruction[50]
```

3.5 Control Flow Overview

3.5.1 Main Loop Sequence

1. **Input Scanning**: Poll button states continuously
2. **Event Detection**: Identify button press/release
3. **Input Decoding**: Convert 4-bit input to16 action
4. **Game Processing**: Execute move or command
5. **State Update**: Modify game state accordingly
6. **Output Generation**: Update display based on new state
7. **AI Consideration**: Computer move in single-player mode

3.5.2 State Management

- **Initialization**: setup() configures hardware and initial state
- **Gameplay**: Main loop handles continuous interaction
- **Special Modes**: Name display, win animations, reset sequences
- **Error Handling**: Input validation and recovery mechanisms

3.6 Inter-Module Communication

3.6.1 Function Call Hierarchy

```
loop()
├── readAllInputs()
├── getinput()
├── getCoordinates()
│   ├── checkGameState()
│   ├── encodeToPins()
│   └── clkexute()
└── computerMove() [if singlePlayer]
```

3.6.2 Global State Dependencies

- **Hardware State**: Pin states synchronized with flip-flop states
- **Game State**: XOMAP array represents current board
- **Display State**: Instruction array drives animations
- **Mode State**: singlePlayer flag alters control flow

3.7 Timing and Synchronization

3.7.1 Critical Timing Operations

- **Clock Pulses**: 10ms minimum width for reliable flip-flop latching
- **Debounce Delays**: 50ms button debouncing with 200ms cooldown
- **Animation Timing**: 200ms blink intervals for visual effects
- **AI Delays**: 500ms pause for computer move visibility

3.7.2 Real-time Considerations

- **Non-blocking Design**: Main loop maintains responsiveness
- **State Preservation**: Game state persists during animations
- **Input Buffering**: Single input processing per loop iteration
- **Mode Transitions**: Smooth switching between game states

This architecture provides a robust foundation for the Tic Tac Toe system, balancing hardware control complexity with maintainable game logic structure.

4. Game Logic Implementation

4.1 Game State Management

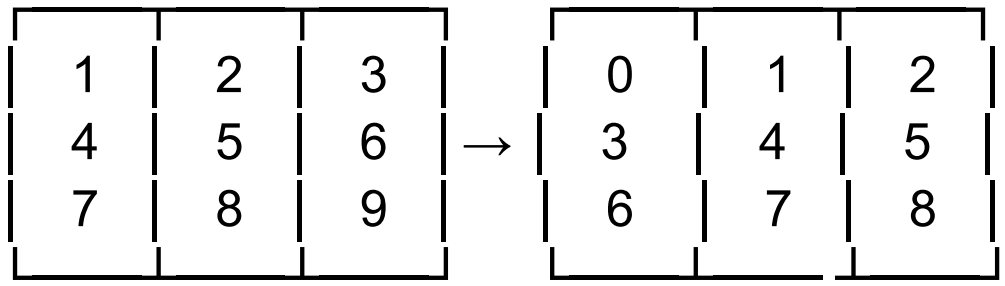
4.1.1 Core Data Structures

```
// Primary game board representation (3x3 grid)
char XOMAP[9] = {
  '-','-','-',   // Row 1: positions 0,1,2
  '-','-','-',   // Row 2: positions 3,4,5
  '-','-','-'    // Row 3: positions 6,7,8
};

// Game control variables
char player = 'x';        // Current player ('x' or 'o')
bool gamedone = false;    // Game completion flag
bool singlePlayer = false; // Game mode selection
```

4.1.2 Board Coordinate System

Logical Mapping (1-9) → Array Index (0-8):



4.2 Move Processing Pipeline

4.2.1 Input to Action Conversion

```
void getCoordinates(int cell_num) {
    // Special command: Reset game (all buttons pressed = 15)
    if (cell_num == 15) {
        fullReset(); // Reset game hardware read
        return;
    }
    // Special command: Display name (button combination = 14)
    else if (cell_num == 14) {
        displayName();
        return;
    }
    // Special command: Toggle single player mode (button combination = 13)
    else if (cell_num == 13) {
        if (singlePlayer == true)
            singlePlayer = false; // Switch to two-player
        else
            singlePlayer = true;  // Switch to single-player
        return;
    }
```

```
else if (cell_num > 0 && cell_num < 10) {
    if (XOMAP[cell_num - 1] == '-') { // Check if cell is empty
        XOMAP[cell_num - 1] = player; // Place player's mark

        // Check game state after move
        char gameState = checkGameState();

        // Handle X win
        if (gameState == 'x') {
            if (gamedone == false) {
                encodeToPins(cell_num); // Show winning move
                clkexute();
                gamedone = true;
            }
            xWin(); // Start win animation
        }
        // Handle O win
        else if (gameState == 'o') {
            if (gamedone == false) {
                encodeToPins(cell_num); // Show winning move
                clkexute();
                gamedone = true;
            }
            oWin(); // Start win animation
        }
        // Handle draw
        else if (gameState == 'd') {
            if (gamedone == false) {
                encodeToPins(cell_num); // Show final move
                clkexute();
                gamedone = true;
            }
            gameDraw(); // Start draw animation
        }
        // Game continues
        else {
            // Normal move - update display
            encodeToPins(cell_num);
            clkexute();
```

```
        // Game continues
        else {
            // Normal move - update display
            encodeToPins(cell_num);
            clkexute();

            // SINGLE PLAYER MODE: Computer makes move
            if (singlePlayer && !gamedone) {
                delay(500); // Pause so player can see their move
                computerMove(); // AI makes its move

                // Check if computer won
                gameState = checkGameState();
                if (gameState == 'o') {
                    oWin(); // Computer wins
                }
                else if (gameState == 'd') {
                    gameDraw(); // Game tied
                }
            }
```

4.2.2 Move Validation & Execution

```
// Game move: Cells 1-9 for Tic Tac Toe
else if (cell_num > 0 && cell_num < 10) {
    if (XOMAP[cell_num - 1] == '-') { // Check if cell is empty
        XOMAP[cell_num - 1] = player; // Place player's mark
```

```
    // Check game state after move
    char gameState = checkGameState();
```

4.3 Win Condition Detection

4.3.1 Row Victory Detection

```
// Check all 3 rows for win
for (int i = 0; i < 3; i++) {
    if (XOMAP[i * 3] != '-' && XOMAP[i * 3] == XOMAP[i * 3 + 1] && XOMAP[i * 3] == XOMAP[i * 3 + 2]) {
        // Store winning cells for animation
        Instruction[0] = i * 3 + 1;     // First cell in winning row
        Instruction[1] = i * 3 + 2;     // Second cell in winning row
        Instruction[2] = i * 3 + 3;     // Third cell in winning row
        instructionCount = 3;
        winPlayer = XOMAP[i * 3];       // Store winner
        return winPlayer;
    }
}
```

4.3.3 Column Victory Detection

```
// Check all 3 columns for win
for (int i = 0; i < 3; i++) {
    if (XOMAP[i] != '-' && XOMAP[i] == XOMAP[i + 3] && XOMAP[i] == XOMAP[i + 6]) {
        // Store winning cells for animation
        Instruction[0] = i + 1;       // Top cell in column
        Instruction[1] = i + 4;       // Middle cell in column
        Instruction[2] = i + 7;       // Bottom cell in column
        instructionCount = 3;
        winPlayer = XOMAP[i];
        return winPlayer;
    }
}
```

4.3.4 Diagonal Victory Detection

```
// Check both diagonals for win
if (XOMAP[0] != '-' && XOMAP[0] == XOMAP[4] && XOMAP[0] == XOMAP[8]) {
    // Top-left to bottom-right diagonal
    Instruction[0] = 1; // Top-left
    Instruction[1] = 5; // Center
    Instruction[2] = 9; // Bottom-right
    instructionCount = 3;
    winPlayer = XOMAP[0];
    return winPlayer;
}
if (XOMAP[2] != '-' && XOMAP[2] == XOMAP[4] && XOMAP[2] == XOMAP[6]) {
    // Top-right to bottom-left diagonal
    Instruction[0] = 3; // Top-right
    Instruction[1] = 5; // Center
    Instruction[2] = 7; // Bottom-left
    instructionCount = 3;
    winPlayer = XOMAP[2];
    return winPlayer;
}
```

4.3.5 Draw Game Detection or Continue

```
    // Check for draw (board full with no wir
    bool boardFull = true;
    for (int i = 0; i < 9; i++) {
        if (XOMAP[i] == '-') {
            boardFull = false;
            break;
        }
    }

    if (boardFull) {
        return 'd'; // Draw game
    }

    return 'c'; // Continue playing
}
```

4.4 Game State Transitions

4.4.1 State Transition Handling

```
    // Handle X win
    if (gameState == 'x') {
        if (gamedone == false) {
            encodeToPins(cell_num); // Show winning move
            clkexute();
            gamedone = true;
        }
        xWin(); // Start win animation
    }
    // Handle O win
    else if (gameState == 'o') {
        if (gamedone == false) {
            encodeToPins(cell_num); // Show winning move
            clkexute();
            gamedone = true;
        }
        oWin(); // Start win animation
    }
    // Handle draw
    else if (gameState == 'd') {
        if (gamedone == false) {
            encodeToPins(cell_num); // Show final move
            clkexute();
            gamedone = true;
        }
        gameDraw(); // Start draw animation
    }
    // Game continues
```

4.5 Game Mode Management

4.5.1 Single Player Toggle

```
}
// Special command: Toggle single player mode (button combination = 13)
else if (cell_num == 13) {
    if (singlePlayer == true)
        singlePlayer = false; // Switch to two-player
    else
        singlePlayer = true;  // Switch to single-player
    return;
}
```

4.5.2 Game Reset Logic

```
// FULL RESET - Completely reset game state and hardware
void fullReset() {
    // Clear game board
    for (int i = 0; i < 9; i++) {
        XOMAP[i] = '-';
    }
    player = 'x';              // Reset to player X
    gamedone = false;          // Game can start

    // Clear instruction system
    instructionCount = 0;
    for (int i = 0; i < 50; i++) Instruction[i] = -1;
    winPlayer = ' ';

    // Reset hardware flip-flops
    digitalWrite(reset_ff, HIGH);
    delay(10);
    digitalWrite(reset_ff, LOW);
}
```

4.6 Input Validation & Error Prevention

4.7.1 Move Validation Rules

- **Range Checking**: Only cells 1-9 accepted as valid moves
- **Availability Check**: Prevents overwriting occupied cells
- **Game State Awareness**: Blocks moves after game completion
- **Input Sanitization**: Handles invalid 4-bit combinations gracefully

4..2 State Integrity

- **Atomic Operations**: Move execution and state update are atomic
- **Consistency Checks**: Board state always matches display state
- **Race Condition Prevention**: Single input processing per loop iteration
- **Mode Transition Safety**: Clean state transitions between game phases

5. Input Control Architecture

5.1 Hardware Input Architecture

5.1.1 Physical Input Configuration

- **15 Push Buttons**: Game controls and function inputs
- **Hardware Encoder**: 28 OR gates for 16-to-4 bit compression
- **4-bit Input Lines**: Arduino pins 2-5 receive encoded data
- **Pull-down Resistors**: 10kΩ to ground for stable LOW state
- **Active-HIGH Logic**: Buttons pull lines HIGH when pressed

5.1.2 Input Pin Mapping

```
const int BUTTONS[] = { 2, 3, 4, 5 }; // 4-bit input buttons A,B,C,D
const int NUM_BUTTONS = 4;
```

```
// Configure INPUT pins with pull-up resistors
for (int i = 0; i < NUM_BUTTONS; i++) {
    pinMode(BUTTONS[i], INPUT);
}
```

5.2 Input Processing Pipeline

5.2.1 Continuous Scanning

Main loop constantly polls button states through `readAllInputs()`

```
// READ ALL INPUTS - Ecoded Read current state of all 15 buttons
void readAllInputs(bool inputs[]) {
    for (int j = 0; j < NUM_BUTTONS; j++) {
        inputs[j] = (digitalRead(BUTTONS[j]) == HIGH); // true if pressed
    }
}
```

5.2.2 Signal Processing Flow

1. **Initial Detection**: Any button press triggers processing
2. **Debounce Delay**: 50ms wait for signal stabilization
3. **Release Wait**: Processing continues after button release
4. **Final Read**: Capture stable 4-bit input state
5. **Command Execution**: Route to appropriate handler

```
// Check each button for press events
for (int i = 0; i < NUM_BUTTONS; i++) {
    if (currentInputs[i] == true) {
        // Button pressed - debounce delay
        delay(50);

        // Wait for button release
        while (digitalRead(BUTTONS[i]) == LOW) {
            delay(10);
        }

        // Read final input state after release
        readAllInputs(currentInputs);
        // Convert 4-bit input to decimal cell number
        int cell = getinput(currentInputs[0], currentInputs[1], currentInputs[2], currentInputs[3]);
        // Process the input
        getCoordinates(cell);

        // Prevent multiple reads from same press
        delay(200);
        break; // Process only one button per loop
    }
}
```

5.3 Input Decoding System

5.3.1 Binary to Decimal Conversion

`getinput()` function converts 4-bit combinations to values 0-15

5.3.2 Valid Input Ranges

- **0**: No input (ignored)
- **1-9**: Game board positions
- **13-15**: System commands
- **10-12**: Currently unused

```
// GET INPUT - Convert 4 button states to decimal number (0-15)
int getinput(bool input_A, bool input_B, bool input_C, bool input_D) {
    // Binary to decimal conversion for 4-bit input
    if (!input_A && !input_B && !input_C && !input_D) return 0;  // 0000
    else if (input_A && !input_B && !input_C && !input_D) return 1;  // 0001
    else if (!input_A && input_B && !input_C && !input_D) return 2;  // 0010
    else if (input_A && input_B && !input_C && !input_D) return 3;  // 0011
    else if (!input_A && !input_B && input_C && !input_D) return 4;  // 0100
    else if (input_A && !input_B && input_C && !input_D) return 5;  // 0101
    else if (!input_A && input_B && input_C && !input_D) return 6;  // 0110
    else if (input_A && input_B && input_C && !input_D) return 7;  // 0111
    else if (!input_A && !input_B && !input_C && input_D) return 8;  // 1000
    else if (input_A && !input_B && !input_C && input_D) return 9;  // 1001
    else if (!input_A && input_B && !input_C && input_D) return 10; // 1010
    else if (input_A && input_B && !input_C && input_D) return 11; // 1011
    else if (!input_A && !input_B && input_C && input_D) return 12; // 1100
    else if (input_A && !input_B && input_C && input_D) return 13; // 1101
    else if (!input_A && input_B && input_C && input_D) return 14; // 1110
    else if (input_A && input_B && input_C && input_D) return 15; // 1111
    else return -1; // Error case
}
```

5.4 Command Processing

5.4.1 System Command Mapping

- **13**: Toggle single-player/two-player mode
- **14**: Activate "YAZAN" name display
- **15**: Full system reset

5.4.2 Game Move Processing

`getCoordinates()` function handles:

- Move validation (cell availability)
- Board state updates
- Game outcome checking
- Display synchronization

5.5 Debouncing and Flow Control

5.5.1 Timing Parameters

- **50ms**: Initial debounce delay
- **200ms**: Post-processing cooldown
- **10ms**: Input check intervals during wait states

5.5.2 Single-Input Enforcement

- One command processed per loop iteration
- Break statement prevents multiple processing
- Sequential input handling maintains game flow

5.6 Error Handling

5.6.1 Invalid Input Protection

- Range checking for all input values
- Cell availability verification
- Game state awareness (blocks moves after game end)

5.6.2 Robust Operation

- Handles hardware encoder edge cases
- Manages simultaneous button presses at hardware level
- Recovers from unexpected input states

This input system provides reliable, debounced button reading with comprehensive command routing and robust error handling.

6. Output and Display System

6.1 Software Control Architecture

6.1.1 Pin Management

- **5-bit Output Control**: Pins A,B,C,D,E for display addressing
- **Clock Signal**: Dedicated CLK pin for data latching
- **Reset Control**: RESET_FF pin for display clearing
- **Synchronized Operations**: Coordinated pin state changes

```
//OUTPUT PINS - Control the external hardware
const int A = 13;         // Bit 0 of 4-bit output (LSB)
const int B = 12;         // Bit 1 of 4-bit output
const int C = 11;         // Bit 2 of 4-bit output
const int D = 10;         // Bit 3 of 4-bit output (MSB)
const int E = 9;          // Player indicator: LOW=X, HIGH=O
const int clk = 8;        // Clock signal to latch the output
const int reset_ff = 7;// Reset pin to clear all flip-flops
```

```
  // Configure OUTPUT pins
  pinMode(A, OUTPUT);       // Data bit 0
  pinMode(B, OUTPUT);       // Data bit 1
  pinMode(C, OUTPUT);       // Data bit 2
  pinMode(D, OUTPUT);       // Data bit 3
  pinMode(E, OUTPUT);       // Player indicator
  pinMode(clk, OUTPUT);     // Clock signal
  pinMode(reset_ff, OUTPUT); // Flip-flop reset
```

6.1.2 Core Display Functions

- **encodeToPins()**: Converts numbers to 4-bit output + player indicator
- **clkexute()**: Generates clock pulse for flip-flop latching
- **resetDisplay()**: Clears all flip-flops via reset signal

```
// CLOCK EXECUTE - Send clock pulse and reset data lines
void clkexute() {
    digitalWrite(clk, HIGH);  // Rising edge latches data
    delay(10);                // Pulse width
    digitalWrite(clk, LOW);   // Falling edge

    // Reset data lines after clock (prepare for next output)
    digitalWrite(A, LOW);
    digitalWrite(B, LOW);
    digitalWrite(C, LOW);
    digitalWrite(D, LOW);
}
```

6.2 Display Data Encoding

6.2.1 Number to Pin Mapping

```
// ENCODE TO PINS - Convert number to 4-bit output and toggle player
void encodeToPins(int number) {
    if (number < 0 || number > 15) return; // Validate input

    // Toggle player only if game is active
    if (gamedone == false)
        if (player == 'x') {
            digitalWrite(E, LOW);   // E=LOW indicates player X
            player = 'o';           // Switch to next player
        }
        else {
            digitalWrite(E, HIGH);  // E=HIGH indicates player O
            player = 'x';           // Switch to next player
        }

    // Output number as 4-bit binary on pins A,B,C,D
    digitalWrite(A, (number & 1) ? HIGH : LOW);   // Bit 0 (1)
    digitalWrite(B, (number & 2) ? HIGH : LOW);   // Bit 1 (2)
    digitalWrite(C, (number & 4) ? HIGH : LOW);   // Bit 2 (4)
    digitalWrite(D, (number & 8) ? HIGH : LOW);   // Bit 3 (8)
}
```

6.2.2 Player Indicator Control

- **E = LOW**: Player X turn/display
- **E = HIGH**: Player O turn/display
- **Automatic Switching**: Toggled in encodeToPins() during gameplay

6.3 Display Sequences

6.3.1 YAZAN Name Display

- **Pre-defined Patterns**: yazanSequence array stores cell activations
- **Sequential Execution**: Loops through letter patterns with delays
- **Visual Effects**: Alternating E pin for animation effect
- **Timing Control**: 2000ms pauses between letters

6.3.2 Game State Display

- **Move Display**: Immediate update after valid moves
- **Win Patterns**: Instruction array stores winning cell sequences
- **Animation Control**: instructionCount manages active patterns

6.4 Timing and Synchronization

6.4.1 Critical Delays

- **Clock Pulse**: 10ms width for reliable flip-flop operation
- **Reset Pulse**: 10ms duration for complete clearing
- **Animation Timing**: 200ms intervals for blinking effects
- **Display Updates**: Immediate execution after game actions

6.4.2 State Management

- **Display State**: Mirrors internal XOMAP board state
- **Player Synchronization**: E pin matches current player variable
- **Reset Coordination**: Hardware and software state reset together

6.5 Pattern Generation

6.5.1 Static Patterns

- **Cell Numbers 1-9**: Direct position mapping
- **Special Commands**: Values 13-15 for system functions
- **Error Handling**: Range validation in encodeToPins()

6.5.2 Dynamic Patterns

- **Win Animations**: Blinking sequences from Instruction array
- **Text Display**: Sequential pattern playback for "YAZAN"
- **Game State Visuals**: Different patterns for X win, O win, draw

This software-controlled display system provides abstracted hardware control while maintaining precise timing and synchronization for reliable visual feedback.

7. Artificial Intelligence Module

7.1 AI System Overview

7.1.1 Purpose and Role

- **Computer Opponent**: Provides challenging gameplay in single-player mode
- **Strategic Decision Making**: Implements Tic Tac Toe game theory
- **Adaptive Difficulty**: Adjusts strategy based on game phase
- **Non-deterministic Behavior**: Incorporates randomness for varied gameplay

7.1.2 Integration Points

- **Trigger Condition**: Activated after human moves in single-player mode
- **Timing Control**: 500ms delay for natural gameplay flow
- **State Awareness**: Operates within current game board context

7.2 Decision Making Architecture

7.2.1 Strategic Priority System

The AI employs a hierarchical decision-making process:

1. **Winning Move**: Complete three-in-a-row if possible
2. **Blocking Move**: Prevent opponent from winning
3. **Center Control**: Take center position if available
4. **Corner Control**: Occupy corner positions
5. **Random Selection**: Choose any available cell

7.2.2 Game Phase Detection

- **Early Game** (moves 0-1): Random placement for variety
- **Mid/Late Game** (moves 2+): Strategic decision making
- **Move Counting**: Tracks occupied cells to determine phase

7.3 AI Algorithm Implementation

7.3.1 computerMove() Function

Central AI entry point that coordinates all decision logic:

- **Phase Detection**: Analyzes current board state
- **Strategy Selection**: Chooses appropriate tactical approach
- **Move Execution**: Updates board and display

```
// COMPUTER MOVE - AI decides where to play
void computerMove() {
    int move = -1;

    // Count moves to determine game phase
    int moveCount = 0;
    for (int i = 0; i < 9; i++) {
        if (XOMAP[i] != '-') moveCount++;
    }

    // Early game: random moves for variety
    if (moveCount < 2) {
        do {
            move = millis() % 9; // Random position
        } while (XOMAP[move] != '-');
    }
    // Mid/late game: strategic moves
    else {
        // Strategy priority: 1. Win, 2. Block, 3. Center, 4. Corner, 5. Random
        move = findWinningMove('o'); // Try to win
        if (move == -1) move = findWinningMove('x'); // Block player win
        if (move == -1) move = takeCenter(); // Take center if available
        if (move == -1) move = takeCorner(); // Take corner if available
        if (move == -1) move = randomEmptyCell(); // Random empty cell
    }

    // Execute the chosen move
    if (move != -1 && XOMAP[move] == '-') {
        XOMAP[move] = 'o';
        encodeToPins(move + 1);
        clkexute();
    }
}
```

7.3.2 findWinningMove()

Searches for immediate win opportunities:

- **Board Simulation**: Tests each empty cell for winning potential
- **Temporary State Modification**: Places mark and checks win condition
- **State Restoration**: Reverts board after simulation
- **Both Players**: Can search for AI wins or human blocks

```
// FIND WINNING MOVE - Check if player can win with one move
int findWinningMove(char player) {
    // Test each empty cell to see if it creates a win
    for (int i = 0; i < 9; i++) {
        if (XOMAP[i] == '-') {
            // Simulate move
            XOMAP[i] = player;
            char result = checkGameState();
            XOMAP[i] = '-'; // Undo simulation

            if (result == player) {
                return i; // This move wins
            }
        }
    }
    return -1; // No winning move found
}
```

7.3.3 Position Preference Functions

- **takeCenter()**: Prioritizes center position (cell 5)

```
// TAKE CENTER - Prefer center position
int takeCenter() {
    if (XOMAP[4] == '-') return 4; // Center cell
    return -1;
}
```

- **takeCorner()**: Selects from available corner positions (1,3,7,9)

```
// TAKE CORNER - Prefer corner positions
int takeCorner() {
    int corners[] = { 0, 2, 6, 8 }; // Four corner positions
    // Find first available corner
    for (int i = 0; i < 4; i++) {
        if (XOMAP[corners[i]] == '-') {
            return corners[i];
        }
    }
    return -1; // No corners available
}
```

- **randomEmptyCell()**: Fallback to any available position

```
// RANDOM EMPTY CELL - Choose any available cell randomly
int randomEmptyCell() {
    // Collect all empty cells
    int emptyCells[9];
    int count = 0;

    for (int i = 0; i < 9; i++) {
        if (XOMAP[i] == '-') {
            emptyCells[count] = i;
            count++;
        }
    }

    // Return random empty cell if available
    if (count > 0) {
        return emptyCells[millis() % count];
    }
    return -1; // No empty cells (shouldn't happen)
}
```

7.4 Move Selection Logic

7.4.1 Win Prevention

- **Immediate Threat Response**: Blocks human winning moves
- **Multiple Threat Detection**: Handles complex board situations
- **Forced Move Recognition**: Identifies critical positions

7.4.2 Strategic Positioning

- **Center Control**: Recognizes center's strategic value (4 winning lines)
- **Corner Advantage**: Understands corner positions (3 winning lines each)
- **Edge Avoidance**: Less preference for edge positions (2 winning lines)

7.5 Randomization and Variety

7.5.1 Non-repetitive Gameplay

- **millis() Based Randomness**: Uses system time for move variation
- **Early Game Randomization**: Different starting positions each game
- **Multiple Option Handling**: Random selection when multiple equal moves exist

7.5.2 Adaptive Behavior

- **Board State Analysis**: Responds to human player's strategy
- **Dynamic Decision Making**: Adjusts based on current game context
- **Error Recovery**: Handles unexpected board states gracefully

7.6 Performance Characteristics

7.6.1 Execution Efficiency

- **Minimal Computation**: Efficient board analysis algorithms
- **Rapid Decision Making**: Fast move calculation for responsive gameplay
- **Memory Efficiency**: Low memory footprint for embedded system

7.6.2 Gameplay Quality

- **Perfect Play Capability**: Never loses when playing optimally
- **Human-like Imperfections**: Occasional suboptimal moves for engagement
- **Balanced Challenge**: Provides appropriate difficulty level

This AI module creates a competent and engaging computer opponent that demonstrates strategic Tic Tac Toe play while maintaining computational efficiency suitable for embedded systems.

8. Animation and Special Effects System

8.1 Animation Framework Architecture

8.1.1 Core Animation Engine

The animation system is built around a centralized instruction-based architecture that manages all visual effects and special sequences:

```
// INSTRUCTION SYSTEM - For win animations and special effects
int Instruction[50] = { -1 }; // Array to store cell patterns for animations
int instructionCount = 0;   // Number of active instructions
char winPlayer = ' ';       // Stores which player won ('x' or 'o')
```

8.1.2 Pattern Generation System

- **Dynamic Pattern Loading**: Instructions array populated based on game outcomes
- **Flexible Sequence Handling**: Supports variable-length animation patterns
- **Multi-Pattern Capability**: Can store complex animation sequences for advanced effects

8.2 Victory Animation System

8.2.1 Win Detection Integration

The animation system tightly integrates with game logic through `checkGameState()`:

- **Automatic Pattern Generation**: Winning lines automatically stored in Instruction array
- **Three-Cell Sequences**: Row, column, and diagonal wins generate 3-cell patterns
- **Immediate Availability**: Patterns ready for animation when win detected

8.2.2 Player X Victory Animation (xWin())

```
// X WIN - Animation for player X victory
void xWin() {
    // Reset hardware and set player indicator
    digitalWrite(reset_ff, HIGH);
    delay(10);
    digitalWrite(reset_ff, LOW);
    delay(100);
    digitalWrite(E, LOW); // X player indicator

    // Blink winning cells 10 times
    for (int blinkCount = 0; blinkCount < 10; blinkCount++) {
        // Turn ON winning cells
        for (int i = 0; i < instructionCount; i++) {
            if (Instruction[i] != -1) {
                encodeToPins(Instruction[i]);
                clkexute();
            }
        }
        delay(200); // ON time

        // Turn OFF all cells
        digitalWrite(reset_ff, HIGH);
        delay(10);
        digitalWrite(reset_ff, LOW);
        delay(200); // OFF time
    }

    fullReset(); // Reset game after animation
}
```

8.2.3 Player O Victory Animation (oWin())

- **Symmetric Structure**: Same timing and cycle count as xWin()
- **Player Differentiation**: E pin set HIGH for O player indication
- **Consistent Visual Language**: Maintains same blink pattern for both players

```
// O WIN - Animation for player O victory
void oWin() {
    // Reset hardware and set player indicator
    digitalWrite(reset_ff, HIGH);
    delay(10);
    digitalWrite(reset_ff, LOW);
    delay(100);
    digitalWrite(E, HIGH); // O player indicator

    // Blink winning cells 10 times
    for (int blinkCount = 0; blinkCount < 10; blinkCount++) {
        // Turn ON winning cells
        for (int i = 0; i < instructionCount; i++) {
            if (Instruction[i] != -1) {
                encodeToPins(Instruction[i]);
                clkexute();
            }
        }
        delay(200); // ON time

        // Turn OFF all cells
        digitalWrite(reset_ff, HIGH);
        delay(10);
        digitalWrite(reset_ff, LOW);
        delay(200); // OFF time
    }

    fullReset(); // Reset game after animation
}
```

8.3 Draw Game Animation

8.3.1 Full Board Activation

```
// GAME DRAW - Animation for tied game
void gameDraw() {
    // Set instructions to blink all 9 cells
    for (int i = 0; i < 9; i++) Instruction[i] = i + 1;
    instructionCount = 9;

    // Reset hardware
    digitalWrite(reset_ff, HIGH);
    delay(10);
    digitalWrite(reset_ff, LOW);
    delay(100);

    // Blink all cells 10 times
    for (int blinkCount = 0; blinkCount < 10; blinkCount++) {
        // Turn ON all cells with correct player indicators
        for (int i = 0; i < instructionCount; i++) {
            // Set E pin based on which player occupied each cell
            if (XOMAP[i] == 'x')
                digitalWrite(E, LOW);  // X's cells
            else
                digitalWrite(E, HIGH); // O's cells

            if (Instruction[i] != -1) {
                encodeToPins(Instruction[i]);
                clkexute();
            }
        }
        delay(200); // ON time

        // Turn OFF all cells
        digitalWrite(reset_ff, HIGH);
        delay(10);
        digitalWrite(reset_ff, LOW);
        delay(200); // OFF time
    }

    fullReset(); // Reset game after animation
}
```

8.3.2 Winner Preservation

- **Historical State Maintenance**: Remembers original player assignments for each cell
- **Visual Distinction**: Maintains X/O color coding during draw animation
- **State Accuracy**: Faithfully represents the final game board

8.4 Special Effects Sequences

8.4.1 YAZAN Name Display System

```
// YAZAN LETTER PATTERNS - Display name on 3x3 grid
// Each letter defined by lit cells, 0=separator between letters
int yazanSequence[] = {
    // Y: Cells 1,3,5,8 form letter Y
    1, 3, 5, 8, 0,
    // A: Cells 2,4,5,6,7,9 form letter A
    2, 4, 5, 6, 7, 9, 0,
    // Z: Cells 1,2,3,5,7,8,9 form letter Z
    1, 2, 3, 5, 7, 8, 9, 0,
    // A: Cells 2,4,5,6,7,9 form letter A
    2, 4, 5, 6, 7, 9, 0,
    // N: Cells 1,3,4,5,6,7,9 form letter N
    1, 3, 4,5, 6, 7, 9, 0
};
```

8.4.2 Name Animation Execution

```
// DISPLAY NAME - Show "YAZAN" letter by letter with delays
void displayName() {
    for (int i = 0; i < 36; i++) { // Loop through sequence
        if (yazanSequence[i] == 0) {
            delay(2000);          // Pause between letters
            // Reset display between letters
            digitalWrite(reset_ff, HIGH);
            delay(10);
            digitalWrite(reset_ff, LOW);
        }
        else {
            // Display cell with alternating E pin (visual effect)
            digitalWrite(E, LOW);
            encodeToPins(yazanSequence[i]);
            clkexute();
            digitalWrite(E, HIGH);
            encodeToPins(yazanSequence[i]);
            clkexute();
        }
    }
    // Final reset after name display
    digitalWrite(reset_ff, HIGH);
    delay(10);
    digitalWrite(reset_ff, LOW);
}
```

8.5 Timing and Synchronization System

8.5.1 Precision Timing Control

- **Blink Cycle Timing**: 200ms ON / 200ms OFF for optimal visual perception
- **Letter Transition Delay**: 2000ms pauses for clear letter separation
- **Hardware Synchronization**: 10ms pulses for reliable flip-flop operation
- **Animation Duration**: 20 cycles (8 seconds) for victory animations

8.5.2 Frame Rate Management

- **Consistent Update Rate**: Maintains smooth animation throughout sequences
- **Hardware Timing Compliance**: Respects flip-flop setup and hold times
- **Visual Comfort**: Timing optimized for human visual perception

8.6 State Management and Transitions

8.6.1 Animation Lifecycle

1. **Initialization**: Hardware reset and pattern loading
2. **Execution**: Looped animation sequence with precise timing
3. **Termination**: Clean reset and return to main game state
4. **Cleanup**: Instruction array reset for next use

8.6.2 Resource Management

- **Memory Efficiency**: Reuses Instruction array for all animation types
- **State Preservation**: Maintains game integrity during animations
- **Hardware Coordination**: Proper reset sequences between animation phases

8.7 Visual Design Principles

8.7.1 User Experience Optimization

- **Clear Visual Feedback**: Immediate recognition of game outcomes
- **Consistent Visual Language**: Same animation style for similar events
- **Progressive Disclosure**: Complex patterns revealed sequentially
- **Attention Guidance**: Winning lines emphasized through blinking

8.7.2 Accessibility Considerations

- **Adequate Timing**: Sufficient duration for pattern recognition
- **Clear Transitions**: Distinct separation between animation states
- **Consistent Patterns**: Predictable animation behavior